1. Cambia test_size a 0.3 en la Práctica 04. ¿Cómo cambian la pendiente y el R^2? Explica por qué.

E9_propuesto1.ipynb

IA_Guia09_Regresion_Lineal > EjerciciosPropuestos9 > E9_propuesto1.ipynb > import numpy as np

Generar | Código | Markdown | Ejecutar todo | Reiniciar | Borrar todas las salidas | Jupyter Variables | Esquema | ...

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.model_selection import train_test_split

# Datos simulados idénticos a la práctica base
rng = np.random.default_rng(0)
horas = rng.uniform(1, 10, size=50).reshape(-1, 1)
nota = 6 + 1.3 * horas.ravel() + rng.normal(0, 1, size=50)

# 1. Modelo con test_size = 0.2 (Original)
X_train20, X_test20, y_train20, y_test20 = train_test_split(
    horas, nota, test_size=0.2, random_state=0
)
reg20 = LinearRegression().fit(X_train20, y_train20)
y_pred20 = reg20.predict(X_test20)

# 2. Modelo con test_size = 0.3 (Modificado)
X_train30, X_test30, y_train30, y_test30 = train_test_split(
    horas, nota, test_size=0.3, random_state=0
)
reg30 = LinearRegression().fit(X_train30, y_train30)
y_pred30 = reg30.predict(X_test30)

print("COMPARACIÓN TEST_SIZE 0.2 VS 0.3")
print(
    f"test_size=0.2 -> Pendiente: {reg20.coef_[0]:.3f} | R2: {r2_score(y_test20, y_pred20):.3f}"
)
print(
    f"test_size=0.3 -> Pendiente: {reg30.coef_[0]:.3f} | R2: {r2_score(y_test30, y_pred30):.3f}"
)
```

[2] ✓ 0.0s

```
COMPARACIÓN TEST_SIZE 0.2 VS 0.3
test_size=0.2 -> Pendiente: 1.375 | R2: 0.946
test_size=0.3 -> Pendiente: 1.420 | R2: 0.917
```

2. Crea un dataset de 30 registros con temperatura (°C) y ventas de helados. Entrena una regresión lineal simple y predice las ventas para 28 °C.

E9_propuesto2.ipynb

IA_Guia09_Regresion_Lineal > EjerciciosPropuesos9 > E9_propuesto2.ipynb > import numpy as np

Generar | Código | Markdown | Ejecutar todo | Reiniciar | Borrar todas las salidas | Jupyter Variables | Esquema

Generar | Código | Markdown

```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Crear dataset de 30 registros
rng = np.random.default_rng(42)
temperatura = rng.uniform(18, 35, size=30).reshape(-1, 1)
# Ventas generadas con relación positiva y ruido
ventas = 50 + 12 * temperatura.ravel() + rng.normal(0, 15, size=30)

df_helados = pd.DataFrame(
    {"temperatura": temperatura.ravel(), "ventas": ventas}
)

# Entrenar regresión lineal simple
modelo_helados = LinearRegression()
modelo_helados.fit(temperatura, ventas)

# Predicción para 28 °C
temp_consulta = np.array([[28]])
venta_predicha = modelo_helados.predict(temp_consulta)[0]

print("PREDICCIÓN DE VENTAS DE HELADOS ")
print(f"Pendiente: {modelo_helados.coef_[0]:.2f}")
print(f"Intercepto: {modelo_helados.intercept_:.2f}")
print(f"Ventas estimadas para 28 °C: {venta_predicha:.2f} unidades")
```

[2] ✓ 0.0s

```
PREDICCIÓN DE VENTAS DE HELADOS
Pendiente: 11.87
Intercepto: 55.28
Ventas estimadas para 28 °C: 387.64 unidades
```

3. En la Práctica 05 agrega la variable distrito_premium (0 o 1) al cálculo del precio. ¿Mejora el R^2?

```
)

# Modelo 1: Sin distrito_premium
X1 = df_casas[["m2", "cuartos"]]
y = df_casas["precio"]
X1_train, X1_test, y1_train, y1_test = train_test_split(
    X1, y, test_size=0.2, random_state=42
)

reg1 = LinearRegression().fit(X1_train, y1_train)
r2_modelo1 = r2_score(y1_test, reg1.predict(X1_test))

# Modelo 2: Con distrito_premium
X2 = df_casas[["m2", "cuartos", "distrito_premium"]]
X2_train, X2_test, y2_train, y2_test = train_test_split(
    X2, y, test_size=0.2, random_state=42
)

reg2 = LinearRegression().fit(X2_train, y2_train)
r2_modelo2 = r2_score(y2_test, reg2.predict(X2_test))

print(" EVALUACIÓN DE INCLUSIÓN DE VARIABLE PREMIUM ")
print(f"R2 Sin 'distrito_premium': {r2_modelo1:.4f}")
print(f"R2 Con 'distrito_premium': {r2_modelo2:.4f}")
print(f"¿Mejora el R2?: {'Sí' if r2_modelo2 > r2_modelo1 else 'No'}")
```

[2] ✓ 0.0s

```
 EVALUACIÓN DE INCLUSIÓN DE VARIABLE PREMIUM
R2 Sin 'distrito_premium': 0.7246
R2 Con 'distrito_premium': 0.9497
¿Mejora el R2?: Sí
```

E9_propuesto3.ipynb

IA_Guia09_Regresion_Lineal > EjerciciosPropuesos9 > E9_propuesto3.ipynb > import numpy as np

Generar | + Código | + Markdown | Ejecutar todo | Reiniciar | Borrar todas las salidas | Jupyter Variables | Esquema ...

Generar | + Código | + Markdown

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
from sklearn.model_selection import train_test_split

# Datos simulados de viviendas
rng = np.random.default_rng(10)
n_casas = 100
m2 = rng.uniform(50, 200, n_casas)
cuartos = rng.integers(1, 5, n_casas)
distrito_premium = rng.choice([0, 1], size=n_casas, p=[0.7, 0.3])

# Precio depende de m2, cuartos y distrito_premium
precio = (
    50000
    + 1200 * m2
    + 15000 * cuartos
    + 45000 * distrito_premium
    + rng.normal(0, 10000, n_casas)
)

df_casas = pd.DataFrame(
    {
        "m2": m2,
        "cuartos": cuartos,
        "distrito_premium": distrito_premium,
        "precio": precio,
    }
)
```

4. Modifica la Práctica 06 para pedir las horas con input() y mostrar un mensaje si el valor no está entre 0 y 12.

E9_propuesto4.ipynb

IA_Guia09_Regresion_Lineal > EjerciciosPropuesos9 > E9_propuesto4.ipynb > import numpy as np

Generar | Código | Markdown | Ejecutar todo | Reiniciar | Borrar todas las salidas | Jupyter Variables | Esquema

```
import numpy as np
from sklearn.linear_model import LinearRegression

# Entrenar un modelo base previo para ejecutar la validación
rng = np.random.default_rng(0)
horas_base = rng.uniform(1, 10, size=50).reshape(-1, 1)
nota_base = 6 + 1.3 * horas_base.ravel() + rng.normal(0, 1, size=50)

reg_interactivo = LinearRegression().fit(horas_base, nota_base)

try:
    horas_usuario = float(input("Ingrese la cantidad de horas de estudio (0 - 12): "))

    if 0 <= horas_usuario <= 12:
        nota_predicha = reg_interactivo.predict([[horas_usuario]])[0]
        print(f"Para {horas_usuario} horas de estudio, la nota estimada es: {nota_predicha:.2f}")
    else:
        print("Error: El valor ingresado debe estar comprendido entre 0 y 12.")
except ValueError:
    print("Error: Ingrese un valor numérico válido.")
```

[1] ✓ 35.6s

```
Para 3.0 horas de estudio, la nota estimada es: 9.77
```

Generar | Código | Markdown

5. Compara el MAE de un modelo que siempre predice el promedio de las notas con el MAE de tu regresión. ¿Cuánto mejora?

E9_propuesto5.ipynb

IA_Guia09_Regresion_Lineal > EjerciciosPropuesos9 > E9_propuesto5.ipynb > import numpy as np

Generar | Código | Markdown | Ejecutar todo | Reiniciar | Borrar todas las salidas | Jupyter Variables | Esquema

Generar | Código | Markdown

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.model_selection import train_test_split
# Datos simulados de estudiantes
rng = np.random.default_rng(0)
horas = rng.uniform(1, 10, size=50).reshape(-1, 1)
nota = 6 + 1.3 * horas.ravel() + rng.normal(0, 1, size=50)

X_train, X_test, y_train, y_test = train_test_split(
    horas, nota, test_size=0.2, random_state=0
)
# 1. Regresión Lineal
reg_modelo = LinearRegression().fit(X_train, y_train)
y_pred_reg = reg_modelo.predict(X_test)
mae_regresion = mean_absolute_error(y_test, y_pred_reg)
# 2. Modelo Baseline (Siempre predice el promedio del conjunto de entrenamiento)
promedio_train = y_train.mean()
y_pred_promedio = [promedio_train] * len(y_test)
mae_promedio = mean_absolute_error(y_test, y_pred_promedio)
# Mejora
mejora_absoluta = mae_promedio - mae_regresion
mejora_porcentual = (mejora_absoluta / mae_promedio) * 100

print("COMPARACIÓN DE MAE")
print(f"MAE del modelo basado en Promedio : {mae_promedio:.3f}")
print(f"MAE del modelo Regresión Lineal   : {mae_regresion:.3f}")
print(f"Mejora absoluta en error          : {mejora_absoluta:.3f} puntos")
print(f"Porcentaje de mejora del error    : {mejora_porcentual:.2f}%")
```

[3] ✓ 0.0s

```
COMPARACIÓN DE MAE
MAE del modelo basado en Promedio : 3.602
MAE del modelo Regresión Lineal   : 0.826
Mejora absoluta en error          : 2.776 puntos
Porcentaje de mejora del error    : 77.07%
```